

PROCESAMIENTO DIGITAL DE IMÁGENES

Nombre:

Roberto Misael Reyes Cruz
Rene Paul Sandoval Cuamatzi
Especialidad: IIA

Boleta:

2021301999
2022710089

Fecha de la práctica: 20/09/23

Nombre de la práctica: Calculo del histograma.

Resultados de aprendizaje Propuestos (RAP's)

Comprende del concepto de histograma.

Aplica el algoritmo para la creación de un histograma a partir de una imagen.

Identifica los distintos canales que componen una imagen.

Objetivo

Este programa, desarrollado en Python utilizando la biblioteca PIL y NumPy, tiene como objetivo modificar el canal rojo de una imagen digital mediante la adición de 50 unidades a sus valores de intensidad. Luego, recalcula el histograma del canal rojo modificado y lo muestra en un gráfico. El programa se basa en la capacidad de procesamiento de imágenes para ajustar y analizar la distribución de intensidades en un canal específico de una imagen.

Introducción

Se conocerá el intervalo de números que corresponde al color rojo, verde y azul que ocupa la imagen que se analizará.

Para esto se hará un programa en Python que abra una imagen y que realice la búsqueda de tonos de color rojo, verde y azul. Ya que se tengan esa lista de los tonos respecto al color se hará un análisis con un histograma. Esto para visualizar como se ve los niveles en donde se rige más el tono del 0 al 255.

Desarrollo

- Selecciona una imagen de internet o toma una foto con tu celular
- Carga la imagen en numpy de python
- Obtén los histogramas de los 3 canales y muéstralos
- Suma 50 al canal rojo
- Recalcula el histograma del canal rojo y muéstralo.

Un histograma es la representación gráfica en forma de barras, que simboliza la distribución de un conjunto de datos. Sirven para obtener una "primera vista" general, o panorama, de la distribución de la población, o de la muestra, respecto a una característica, cuantitativa y continua.

Es una herramienta usada para representar una distribución por medio de barras. La altura de la barra está en función de la frecuencia (eje y) y el rango (eje x) de una variable continua.

Nos ofrece un vistazo general del comportamiento de las variables, donde logramos analizar aspectos como distribución, dispersión, aleatoriedad y tendencia.

Generalmente el histograma se usa con variables continuas, aunque también se trabaja con variables discretas. Recordemos que variables continuas son aquellas que pueden tomar cualquier valor, es decir un número real.

Son ejemplo de variables continuas el peso, altura, masa o temperatura. Las variables discretas son aquellas que adoptan un valor entero, por ejemplo, el número de personas, el número de quejas, objetos, etc.

Programa

```
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt

# Función para calcular el histograma de un canal
def calcular_histograma_canal(imagen, canal, bins=256, rango=(0, 256)):
    # Crear un arreglo para el histograma con valores iniciales en cero
    histograma = np.zeros(bins, dtype=np.int)

    # Obtener el canal específico de la imagen
    canal_imagen = imagen[:, :, canal]

    # Iterar sobre los píxeles y actualizar el histograma
    for pixel_valor in canal_imagen.ravel():
        if rango[0] <= pixel_valor < rango[1]:
            histograma[pixel_valor] += 1

    return histograma

# Cargar la imagen con PIL
imagen_pil = Image.open("/content/sample_data/hulk.jpeg")

# Convertir la imagen PIL a un arreglo NumPy
imagen_np = np.array(imagen_pil)

# Crear un arreglo vacío para almacenar los histogramas
histogramas = []

# Usar un bucle for para calcular los histogramas para cada canal
for canal in range(3):
    # Calcular el histograma del canal actual utilizando la función personalizada
    hist = calcular_histograma_canal(imagen_np, canal)

    # Agregar el histograma a la lista de histogramas
    histogramas.append(hist)

# Sumar 50 al canal rojo
imagen_modificada = np.copy(imagen_np)
```

```

imagen_modificada[:, :, 0] = np.clip(imagen_modificada[:, :, 0] + 50, 0, 255)

# Recalcular el histograma del canal rojo modificado utilizando la función personalizada
hist_rojo_modificado = calcular_histograma_canal(imagen_modificada, 0)

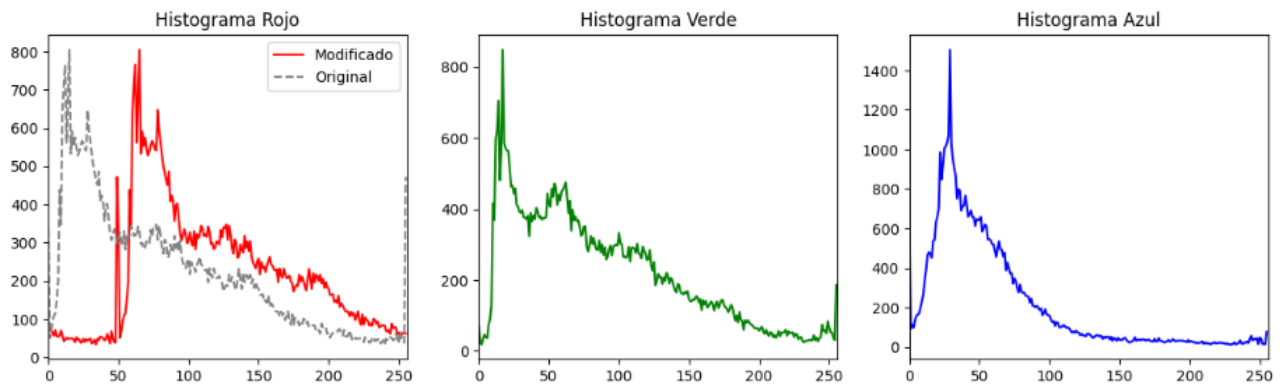
# Mostrar los histogramas
colors = ['red', 'green', 'blue']
channel_names = ['Rojo', 'Verde', 'Azul']

plt.figure(figsize=(15, 4))
for i in range(3):
    plt.subplot(1, 3, i + 1)
    plt.title(f"Histograma {channel_names[i]}")
    if i == 0:
        plt.plot(np.arange(256), hist_rojo_modificado, color=colors[i], label="Modificado")
        plt.plot(np.arange(256), histogramas[i], color='gray', linestyle='--', label="Original")
        plt.legend()
    else:
        plt.plot(np.arange(256), histogramas[i], color=colors[i])
    plt.xlim([0, 256])

plt.show()

```

Resultado



Imagen



Descripción detallada del programa:

- a. Carga de la imagen: El programa comienza cargando una imagen desde un archivo utilizando la biblioteca PIL. La imagen se convierte en un arreglo NumPy para su posterior procesamiento.
- b. Función para calcular el histograma de canal:
- c. Se utiliza una función previamente definida llamada `calcular_histograma_canal` para calcular el histograma del canal rojo de la imagen original. Esta función recorre los píxeles del canal rojo y cuenta cuántos píxeles tienen cada valor de intensidad.
- d. Modificación del canal rojo:
- e. Se suma un valor de 50 unidades a todos los píxeles del canal rojo de la imagen. Esto tiene el efecto de aumentar la intensidad del canal rojo en toda la imagen.
- f. Recálculo del histograma del canal rojo modificado:
- g. Luego de modificar el canal rojo, se utiliza nuevamente la función `calcular_histograma_canal` para calcular el histograma del canal rojo modificado. Esto proporciona información sobre la nueva distribución de intensidades en el canal rojo.

h. Visualización del histograma modificado:

Finalmente, el programa muestra el histograma modificado del canal rojo en un gráfico utilizando la biblioteca Matplotlib. El gráfico incluye etiquetas y rangos apropiados en el eje x para representar los valores de intensidad desde 0 hasta 255.

Conclusiones

El programa proporciona una herramienta sencilla para modificar y analizar el canal rojo de una imagen digital. Al sumar 50 unidades a los valores de intensidad del canal rojo y recalcularlo su histograma, los usuarios pueden observar cómo cambia la distribución de intensidades en el canal. Esto puede ser útil en situaciones donde se requiere un ajuste de color específico o en análisis de imágenes donde se necesita comprender mejor la información contenida en el canal rojo.

Lo que aprendí fue que en este programa es útil en aplicaciones de procesamiento de imágenes donde se requiere ajustar manualmente los canales de color, como corrección de color, mejora de la exposición y realce de características específicas en una imagen.

Bibliografía

<https://github.com/DavidGJ2002/Practica1.2Vision/blob/main/Practica1-2.py>

"Digital Image Processing" by González and Woods", Autor: Rafael C. González y Richard E. Woods
ISBN: 978-0133356724

<https://numpy.org/doc/stable/user/quickstart.html>